Mushroom classification using neural networks

Aymeric Le Riboter - Erwann Lesech

Mai 2025
Promotion 2026

Abstract

Mushroom poisoning poses a notable public health concern, with nearly 2,000 cases reported in France over six months in 2022 [1]. This situation encourages the development of reliable tools for detecting mushroom toxicity, especially during the picking season when identification is difficult due to species diversity and the lack of accessible methods [3]. Existing approaches, including traditional machine learning and deep learning applied to images, face challenges in terms of accuracy, generalization, and practical usability.

To address these challenges, this work proposes a classification approach based on deep neural networks [5] applied to structured tabular data, utilizing categorical features processed through one-hot encoding. Our contribution lies in designing lightweight and efficient neural network architectures that combine the speed of classical models with the expressive capabilities of deep learning, optimized for practical use in mobile applications. Extensive experiments on a balanced mushroom dataset show that our models achieve perfect accuracy and F1 scores, outperforming traditional methods and offering a robust, data-efficient alternative for real-time prediction of mushroom toxicity. These results indicate the potential of the proposed solution to help reduce misclassification risks in sensitive situations. Future work will focus on data augmentation and collection, hybrid models incorporating visual data, and adding probabilistic estimates to enhance user confidence in model predictions.

Contents

Chapter 1

Introduction

Between July 1st and December 31st, 2022, nearly 2,000 cases of mushroom poisoning were reported to French Poison Control Centers [1]. This significant number highlights the importance of developing better tools for preventing such risks, particularly during the mushroom-picking season in autumn. Mushroom recognition is particularly difficult due to the high variability among species and the lack of robust and accessible identification techniques.

The core problem lies in developing a model capable of accurately distinguishing between toxic and non-toxic mushrooms. While various approaches have been proposed in recent years, detailed discussion of these will be presented in the next chapter. Generally, existing methods face limitations in precision, generalization, or usability in real-life conditions [3]. Our objective is to propose a solution that addresses these challenges by combining accuracy, speed, and user accessibility.

Our contribution consists in designing a classification model based on machine learning, particularly using neural networks, to detect mushroom toxicity. Although such models require substantial computational resources for training, they are lightweight and fast once deployed, making them ideal for integration into mobile applications. This would allow real-time use during outdoor activities such as foraging, thereby improving safety for the general public.

This report is organized as follows. Chapter 2 provides an overview of the current state of the art in mushroom toxicity detection. Chapter 3 describes the proposed model and the methodology adopted. Chapter 4 presents the evaluation of the model through experimental results. Finally, Chapter 5 concludes the document and outlines potential future work.

Chapter 2

State of the Art in Mushroom Toxicity Detection

The detection of mushroom toxicity is a major public health concern, particularly due to the severe risks associated with the ingestion of poisonous species. Over the past decades, researchers have attempted to automate this task using various artificial intelligence methods, ranging from traditional machine learning techniques to advanced deep learning models.

Traditional Machine Learning Methods

Early approaches to mushroom classification relied on classical machine learning algorithms applied to structured datasets describing morphological characteristics (e.g., cap shape, gill color, odor). Algorithms such as decision trees, support vector machines (SVM), k-nearest neighbors (KNN), and simple artificial neural networks have been successfully used on well-known datasets like the *Mushroom* dataset from the UCI Machine Learning Repository [2].

These methods are computationally efficient and effective when the data is well-structured. However, they face limitations when dealing with ambiguous or missing attributes and may struggle to capture complex interactions between features, which are often crucial in biological classification tasks.

Deep Learning-Based Approaches

The rise of deep learning has introduced models capable of learning abstract representations from raw data. In the context of mycology, convolutional neural networks (CNNs) have been used to analyze mushroom images. These models, such as AlexNet, ResNet, and YOLO, have demonstrated the ability to capture subtle visual cues related to texture, shape, and color, thus improving classification accuracy [5].

Although these results are promising (with accuracy reaching 80% in certain studies), the effectiveness of such models is highly dependent on the quality and diversity of the image data. Moreover, deep learning models typically require significant computational resources and large annotated datasets, which are not always readily available in this domain.

Mobile Applications and Public Tools

In parallel with academic research, several mobile applications have been developed to help the general public identify mushrooms through image analysis. These applications often rely on artificial intelligence to assess edibility and provide recommendations. However, studies have shown that such tools can produce high error rates, especially when photos are taken under uncontrolled conditions (e.g., poor lighting, unusual angles, or background clutter) [3]. This makes them unreliable when used without expert supervision, potentially putting users at serious health risks.

General Limitations and Recent Trends

Despite technological advances, there remain multiple challenges. Classical models are limited by the representativeness of the training data, while deep learning models require high computational power and extensive data preparation. Furthermore, the natural variability of mushrooms, including their ability to mutate or hybridize, complicates the task of automated classification, especially under real-world conditions.

Proposed Contribution

This work proposes a novel approach to the classification of mushroom toxicity by applying **deep neural networks to tabular data**. Positioned between traditional techniques and modern deep learning, this method aims to combine the interpretability and efficiency of classic models with the expressive power of neural networks in modeling complex feature interactions.

Inspired by a peer-reviewed research protocol, our implementation focuses on achieving high classification accuracy using structured data, without relying on image-based models. The objective is to improve robustness to input variability while ensuring that the method remains computationally accessible.

This contribution is particularly relevant in critical scenarios where a misclassification can leads to serious health consequences. By minimizing false positives and false negatives through careful model design and evaluation, our approach aims to offer a reliable, data-efficient alternative to existing solutions.

Chapter 3

Methodology

This chapter presents the methodology adopted to address the problem of mushroom toxicity classification using neural networks on tabular data. We begin by formally defining the problem, followed by a detailed description of the model architecture, and conclude with the chosen loss function and its justification.

3.1 Problem Formalization

The task at hand is a binary classification problem: given a mushroom sample described by a set of categorical and/or numerical features, the model must predict whether the mushroom is toxic ($y = 1$) or edible ($y = 0$).

Let $\mathbf{x} \in \mathbb{R}^n$ represent a feature vector describing a mushroom, where n is the number of preprocessed input features (after encoding categorical variables). The label $y \in \{0, 1\}$ denotes the ground-truth class corresponding to this input.

Formally, we aim to learn a function:

$$f_\theta : \mathbb{R}^n \to [0, 1]$$

parameterized by θ, such that:

$$\hat{y} = f_\theta(\mathbf{x})$$

where $\hat{y}$ represents the predicted probability that the mushroom is toxic. The decision threshold is typically set to 0.5, so:

$$\text{Class} = \begin{cases} 1 & \text{if } \hat{y} \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

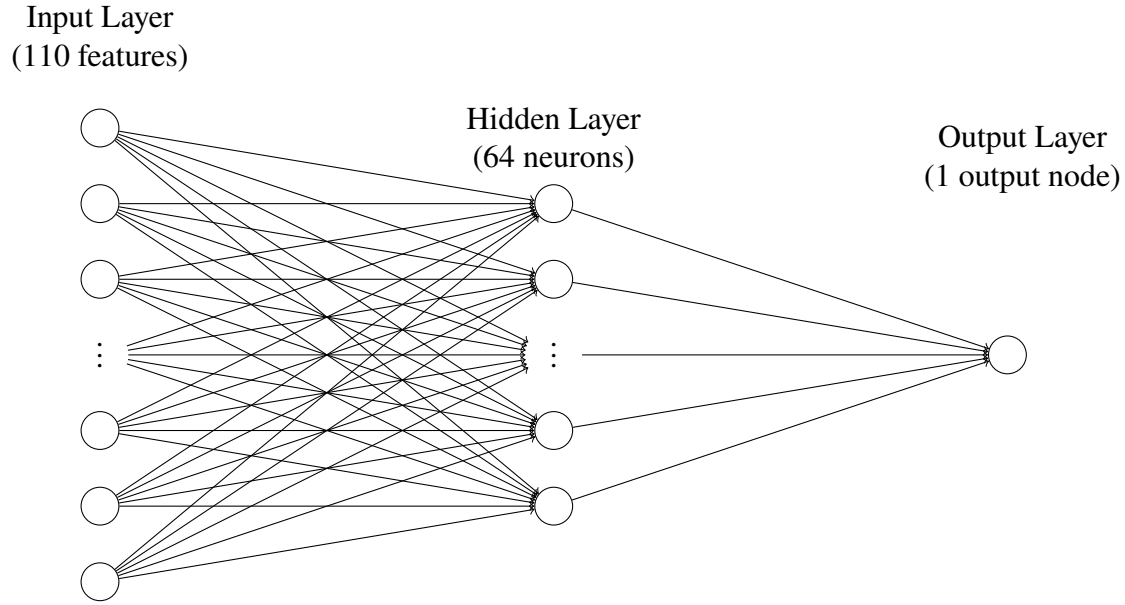
The dataset used for this task is a tabular dataset preprocessed using one-hot encoding, leading to an expanded input dimensionality of 110 features. Details of the dataset will be discussed in the next chapter.

3.2 Model Architecture

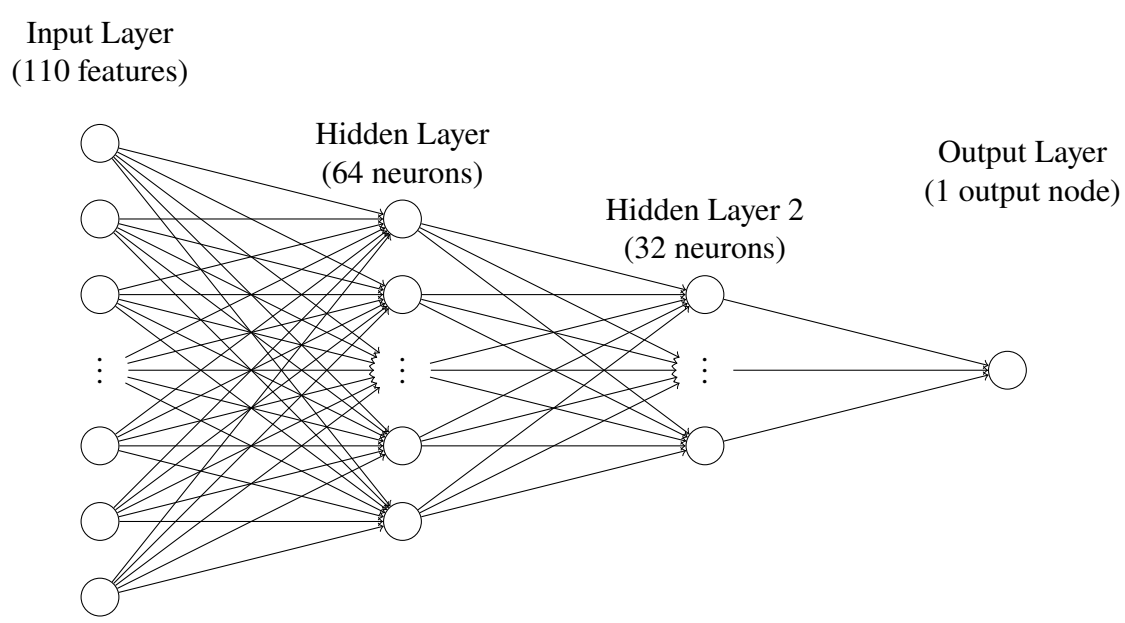
We explore four distinct neural network models by combining two architectural designs with two activation functions in the hidden layers. Each model includes a fully connected architecture designed to work efficiently with structured tabular data.

Architectures

- **Model 1: Shallow network**



- **Model 2: Deeper network**



Each architecture is implemented with two different activation functions in the hidden layers:

- **ReLU (Rectified Linear Unit)**: Widely used due to its computational efficiency and ability to mitigate the vanishing gradient problem.

$$\text{ReLU}(x) = max(0, x)$$

- **SoftPlus**: A smooth approximation of ReLU, which can help with optimization by avoiding hard zero gradients.

$$\text{SoftPlus}(x) = log(1 + e^x)$$

The output layer of all models uses the **sigmoid** activation function to map the output to a probability in the $[0, 1]$ range, appropriate for binary classification.

3.3 Loss Function

Given the binary nature of the classification task, we use the **Binary Cross-Entropy** (BCE) loss function, defined as follows:

$$\mathcal{L}_{BCE}(y, \hat{y}) = -\frac{1}{n} \sum_{i=1}^{n} [y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)]$$

where:

- n is the total number of samples in the dataset.
- y_i is the true label for the i-th sample in the dataset (where $y_i \in \{0, 1\}$),
- $\hat{y}_i$ is the predicted probability for the i-th sample,

This loss function is the standard for binary classification and is particularly well-suited to models where the final output is a sigmoid-activated probability.
We also considered the Hinge Loss function, which is often used in support vector machines. However, it is less appropriate in this context, as our models are probabilistic in nature and require smooth loss landscapes for backpropagation.

Chapter 4

Experiments and Results

4.1 Presentation of the Data

In this section, we describe the dataset used, its characteristics, and the preprocessing steps applied to make the data suitable for training our machine learning models.

4.1.1 Dataset Description

We used the Mushroom Dataset [4] from Audobon Society Field Guide, which contains information about various characteristics of mushrooms, along with a label indicating whether the mushroom is poisonous or not. The dataset consists of a total of 8124 instances, each containing 22 features.

Feature	Categories
cap_shape	convex, bell, sunken, flat, knobbed, conical
cap_surface	smooth, scaly, fibrous, grooves
cap_color	brown, yellow, white, gray, red, pink, buff, purple, cinnamon, green
odor	pungent, almond, anise, none, foul, creosote, fishy, spicy, musty
gill_attachment	free, attached
gill_spacing	close, crowded
gill_size	narrow, broad
gill_color	black, brown, gray, pink, white, chocolate, purple, red, buff, green, yellow, orange
stalk_shape	enlarging, tapering
stalk_surface_above_ring	smooth, fibrous, silky, scaly
stalk_surface_below_ring	s, f, y, k
stalk_color_above_ring	white, gray, pink, brown, buff, red, orange, cinnamon, yellow
stalk_color_below_ring	white, pink, gray, buff, brown, red, yellow, orange, cinnamon

Table 4.1: Features of the Mushroom Dataset (Part I)

Feature	Categories
veil_type	partial
veil_color	white, brown, orange, yellow
number_of_rings	o, t, n
ring_type	pendant, evanescent, large, flaring, none
spore_print_color	black, brown, purple, chocolate, white, green, orange, yellow, buff
population	scattered, numerous, abundant, several, solitary, clustered
habitat	urban, grasses, meadows, woods, paths, waste, leaves
is_poisonous	1 (poisonous), 0 (non-poisonous)

Table 4.2: Features of the Mushroom Dataset (Part II)

4.1.2 Preprocessing the Data

As the features in the dataset are categorical, we applied One-Hot Encoding to convert them into a numerical format. One-Hot Encoding transforms each categorical feature into a binary vector, where each category is represented by a separate binary feature (0 or 1).

For example the feature `cap_shape` with categories ['convex', 'bell', 'sunken', 'flat', 'knobbed', 'conical'] is converted into six binary features, one for each category, with 1 indicating the presence of the category and 0 indicating its absence.

This transformation ensures that our machine learning models can handle the categorical data correctly, as neural networks require numerical input.

The feature `veil_type` was removed from the dataset because it contains only a single value, making it irrelevant for our decision model.

4.1.3 Dataset Split

The dataset was divided using `train_test_split` from `sklearn.model_selection` with a `random_state = 42` into three subsets to train and evaluate the models:

- **Training set:** 70% of the data, used to train the model.
- **Validation set:** 15% of the data, used to tune the hyperparameters and prevent overfitting.
- **Test set:** 15% of the data, used to evaluate the model's performance.

After applying One-Hot Encoding, the number of features increased to 110 binary features (21 original categorical features with multiple categories each). This transformation provides the model with a comprehensive representation of the categorical variables.

4.1.4 Dataset Summary

- **Total instance**: 8124
- **Features after pre-processing**: 111 binary features
- **Target**: `is_poisonous` (0 for edible, 1 for poisonous)

This dataset is balanced, with 48.20% of the samples being poisonous mushrooms, which eliminates the need for specialized metrics typically required for evaluating imbalanced datasets.

4.2 Optimization and Hyperparameters Tuning

In the next sections, we describe the optimization strategies and the hyperparameter tuning process. We tested several optimizers and hyperparameters to find the best configuration for training our models.

4.2.1 Optimizers

We tested the following optimizers:

- **Stochastic Gradient Descent (SGD)**: A basic yet effective optimizer.
- **SGD with Momentum**: A variant of SGD that helps accelerate convergence by adding momentum.
- **Adam (Adaptive Moment Estimation)**: An adaptive optimizer that adjusts the learning rate based on the gradient's moving average.

We performed a grid search over multiple values of the learning rate, momentum, and batch size to determine the best configuration for each optimizer. These experiments provided us with valuable insights into which optimizer performed best for our problem.

4.2.2 Hyperparameter Tuning

We experimented with various values for the following hyperparameters:

- **Learning Rate**: We tested a range of learning rates from $1e-3$ to 0.1.
- **Batch Size**: We experimented with batch sizes of 50 and 100.

- **Regularization parameters**: We tried a range of regularization parameters between 0 and $1e-3$
- **Number of Layers and Nodes**: We evaluated different neural network architectures by varying the number of layers and the number of nodes in each layer.

The hyperparameter tuning was done using cross-validation to avoid overfitting and to ensure that the models were generalizable. Additionally, to guarantee robust and reliable results, each configuration was tested **three times**, with the average of these three trials being taken as the final result for the configuration.

4.3 Results

In this section, we present the results of the experiments. We evaluate the performance of each model using accuracy, training/validation loss curves, and confusion matrices. These results help us understand the strengths and weaknesses of each architecture and guide further improvements.

4.3.1 Evaluation Metrics

To assess the performance of our models, we employed several well-established classification metrics:

- **Accuracy** measures the proportion of correctly predicted samples (both true positives and true negatives) over the total number of samples. It is calculated as:

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

- **Precision** (also known as Positive Predictive Value) evaluates the proportion of true positive predictions among all positive predictions made by the model. It is particularly important when the cost of false positives is high.

$$\text{Precision} = \frac{TP}{TP+FP}$$

- **Recall** (or Sensitivity, or True Positive Rate) assesses the model's ability to correctly identify all actual positive cases. It is useful when missing a positive case (false negative) is more critical.

$$\text{Recall} = \frac{TP}{TP+FN}$$

- **F1 Score** is the harmonic mean of precision and recall, balancing both metrics into a single score. It is especially valuable when working with imbalanced classes.

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision}+\text{Recall}}$$

- **ROC AUC** (Receiver Operating Characteristic - Area Under Curve) quantifies the model's ability to distinguish between the classes at various threshold settings. A higher ROC AUC indicates better separation capability.
- **PR AUC** (Precision-Recall Area Under Curve) captures the trade-off between precision and recall across different thresholds. This metric is particularly informative in scenarios with class imbalance.

These metrics were computed using the `sklearn.metrics` module.

4.3.2 Model Performance and discussion

To facilitate a clear comparison of model performance, we provide a comprehensive summary of the results obtained from all evaluated models. This enables a thorough understanding of each model's strengths and weaknesses.

For an intuitive and interactive visualization of performance metrics across different runs, we utilized `TensorBoard`. This tool allowed us to display all relevant information and apply filters based on specific criteria to make detailed comparisons.

Illustrative examples of these visualizations can be found in the annexes (see Chapter 6).

To evaluate the impact of different neural network architectures, optimizers, and hyperparameter settings, Table 4.3 summarizes the performance of 24 selected model configurations. For each setup, we present the training and validation loss, along with accuracy and F1-score on the validation set. Best-performing models are highlighted in green, while the weakest results are marked in red for improved readability and analysis.

For clarity reasons, we will call each model with following acronyms:

1. **1HReLU**: The model with one hidden layer using the ReLU activation function
2. **1HSoftPlus**: The model with one hidden layer using the SoftPlus activation function
3. **2HReLU**: The model with two hidden layers using the ReLU activation function
4. **2HSoftPlus**: The model with two hidden layers using the SoftPlus activation function

Table 4.3: Summary of 24 model configurations

Model	Optimizer	LR	Reg	Batch	Train Loss	Val Loss	Accuracy	F1-score
1HReLU	Adam	0.001	0.0	50	$2e-5$	$2e-5$	**1.0**	**1.0**
1HReLU	Adam	0.001	0.0	50	$2e-5$	$2e-5$	**1.0**	**1.0**
1HReLU	SGD	0.1	0.0	50	$9e-5$	$9e-5$	**1.0**	**1.0**
1HReLU	SGD	0.001	0.001	100	0.43	0.44	0.89	0.88
1HReLU	SGD_mom	0.1	0.0	50	$5e-5$	$6e-5$	**1.0**	**1.0**
1HReLU	SGD_mom	0.001	0.0	100	0.051	0.053	0.98	0.98
1HSoftPlus	Adam	0.001	0.0	50	$4e-5$	$4e-5$	**1.0**	**1.0**
1HSoftPlus	Adam	0.001	0.0	50	$4e-5$	$4e-5$	**1.0**	**1.0**
1HSoftPlus	SGD	0.1	0.0	50	0.01	0.01	**1.0**	**1.0**
1HSoftPlus	SGD	0.001	0.0	100	0.56	0.56	0.89	0.87
1HSoftPlus	SGD_mom	0.1	0.0	50	$6e-5$	$6e-5$	**1.0**	**1.0**
1HSoftPlus	SGD_mom	0.001	0.0	100	0.07	0.08	0.98	0.97
2HReLU	Adam	0.001	0.0	50	0.0	**0.0**	**1.0**	**1.0**
2HReLU	Adam	0.001	0.0	50	0.0	**0.0**	**1.0**	**1.0**
2HReLU	SGD	0.1	0.0	50	$1.6e-4$	$1.7e-4$	**1.0**	**1.0**
2HReLU	SGD	0.001	0.0	100	0.65	0.64	0.77	0.67
2HReLU	SGD_mom	0.1	0.0	50	$1e-5$	$1e-5$	**1.0**	**1.0**
2HReLU	SGD_mom	0.001	0.001	100	0.027	0.026	0.99	0.99
2HSoftPlus	Adam	0.001	0.0	50	0.0	0.0	**1.0**	**1.0**
2HSoftPlus	Adam	0.001	0.0	50	0.0	0.0	**1.0**	**1.0**
2HSoftPlus	SGD	0.1	0.0	50	$2.1e-4$	$2.2e-4$	**1.0**	**1.0**
2HSoftPlus	SGD	0.001	0.0	100	0.67	0.67	0.57	0.53
2HSoftPlus	SGD_mom	0.1	0.0	50	$1e-5$	$1e-5$	**1.0**	**1.0**
2HSoftPlus	SGD_mom	0.001	0.001	100	0.055	0.058	0.98	0.98

Note: Green highlights indicate the best values (e.g., maximum accuracy or minimum loss); red highlights the worst observed values for each model. `SGD_mom` refers to Stochastic Gradient Descent with momentum.

As shown in Table 4.3, the model's performance on the validation set reveals several insightful patterns regarding both the architecture and the optimizer used.

Impact of Optimizers One of the key observations from the results is the significant influence of the optimizer on model performance:

- **Adam** consistently provides excellent performance across all models. For all tested architectures (1HReLU, 1HSoftPlus, 2HReLU, 2HSoftPlus), Adam achieved perfect scores in accuracy and F1-score (**1.0**) when using well-tuned hyperparameters. Furthermore, validation losses were extremely low (e.g., $2 \cdot 10^{-5}$, $4 \cdot 10^{-5}$, and even 0.0), suggesting that Adam is highly efficient at minimizing the loss and generalizing well on this dataset. This behavior aligns with theoretical expectations: Adam combines the benefits of both RMSprop and momentum, and is known for its robustness to hyperparameter tuning.

- **SGD (vanilla)** shows much more variable results. While certain configurations (e.g., a high learning rate of 0.1 and batch size 50) led to perfect validation accuracy and F1-score, other configurations (especially those with a learning rate of 0.001) significantly underperformed. In particular, models such as 2HSoftPlus with SGD (LR = 0.001) yielded validation loss as high as 0.67, and accuracy as low as 0.57. This suggests that plain SGD is more sensitive to the choice of hyperparameters and may struggle to escape flat regions or local minima during training.
- **SGD with momentum (SGD_mom)** tends to improve the stability and performance of vanilla SGD. Across all models, configurations with momentum and higher learning rates (e.g., 0.1) often reached perfect accuracy and F1-score. Even in less optimal cases (e.g., lower learning rates), performance remained high, typically around 0.97–0.99. This confirms that the momentum mechanism helps accelerate convergence in relevant directions and dampen oscillations.

Effect of Activation Function and Network Depth Although optimizers played a central role, some patterns related to the architecture are also noteworthy:

- **ReLU vs. SoftPlus**: Models using either ReLU or SoftPlus showed comparable results when properly optimized. However, ReLU tends to perform slightly better in configurations with poor hyperparameters, likely due to its non-saturating gradient, which facilitates faster training.
- **Depth (1 layer vs 2 layers)**: Surprisingly, deeper models (2 hidden layers) did not show a significant advantage over simpler 1-layer networks in this specific binary classification task. This may be attributed to the simplicity of the Mushroom dataset and its linearly separable nature. In fact, 1-layer models with good optimizers like Adam or SGD_mom already achieved perfect scores, suggesting that increasing depth beyond this point brings limited benefit.

From these results, we can conclude that Adam is the most reliable and robust optimizer for this task, consistently yielding the best results across all architectures. SGD requires careful tuning of learning rate and regularization to avoid underfitting or poor convergence. However, adding momentum to SGD significantly improves its stability and performance, often matching Adam when configured properly. Finally, the mushroom dataset appears to be easy to classify for most architectures, but poorly tuned models (especially with plain SGD) can still underperform.

Moreover, given the simplicity of the classification task and the high performance achieved even with relatively shallow architectures, there is little benefit in using very deep networks with many layers. Avoiding excessive architectural complexity also helps maintain fast computation times and reduces the risk of overfitting.

Chapter 5

Conclusion and Future Work

This study addressed the challenge of classifying mushroom toxicity using neural networks applied to structured tabular data. By avoiding reliance on image-based approaches, we explored a domain that remains relatively recent in deep learning research, but holds great promise for real-world applications.

Our methodology was rooted in the design and evaluation of multiple neural network architectures, with a focus on balancing performance and computational efficiency. The results obtained on the Mushroom dataset demonstrated that deep neural networks can effectively learn complex feature interactions from encoded categorical data, achieving high classification accuracy.

From a practical point of view, the lightweight nature of the final models makes them highly suitable for deployment in mobile applications. This enables real-time predictions, even on devices with limited hardware capabilities, and offers a valuable tool for users engaging in outdoor activities such as mushroom foraging.

However, several challenges and limitations remain. The models were trained on a relatively clean and well-labeled dataset; real-world conditions are far more variable. Factors such as ambiguous mushroom appearances, hybrid species, environmental damage, or user error during data input can all impact classification performance. Moreover, while the model generalizes well within the dataset, it may struggle with species not represented in the training set.

Future work could proceed in several directions:

- **Data augmentation and collection**: Gathering more diverse datasets, possibly from multiple sources or field studies, would enhance model robustness and generalization.
- **Hybrid models**: Combining tabular and visual data using multi-modal architectures could leverage the strengths of both representations, especially when users provide images alongside structured inputs.
- **Uncertainty estimation**: Adding probabilistic confidence scores or Bayesian neural networks could help identify when the model is uncertain and prompt users to seek expert guidance.

In conclusion, this project demonstrates the viability of neural networks for structured mushroom classification and lays the foundation for reliable and user-friendly tools in the domain of public health and environmental safety.

Bibliography

[1] Anses. *Intoxications accidentelles par des champignons en France métropolitaine*. https://www.anses.fr/fr/system/files/Toxicovigilance2023VIG0127Ra.pdf.

[2] Tilottama Campus. "Classification Of Mushrooms Using Artificial Neural Network". In: *bioRxiv* (2022). URL: https://www.biorxiv.org/content/10.1101/2022.08.31.505980v1.full.pdf.

[3] A. Claypool. "Using AI to spot edible mushrooms could kill you". In: *The Washington Post* (2024). URL: https://www.washingtonpost.com/technology/2024/03/18/ai-mushroom-id-accuracy/.

[4] Audobon Society Field Guide. *Dataset Mushroom Toxicity*. https://archive.ics.uci.edu/dataset/73/mushroom.

[5] Sivakannan Subramania et al. "Deep Learning Based Detection of Toxic Mushrooms in Karnataka". In: *Science Direct* (2023). URL: https://www.sciencedirect.com/science/article/pii/S1877050924006884.

List of Tables

Chapter 6

Annexes

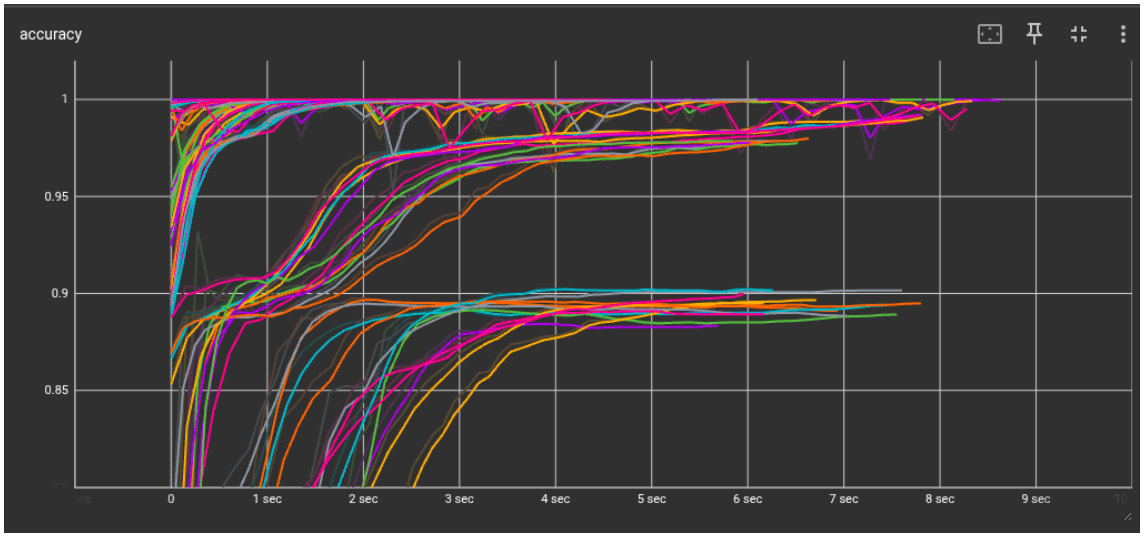


Figure 6.1: Accuracy progression for the 1HSoftPlus model

Figure 6.1 displays the accuracy over training epochs for the model with a single hidden layer using the `Softplus` activation function (referred to as 1HSoftPlus). This metric reflects how well the model correctly classifies input samples as training progresses.

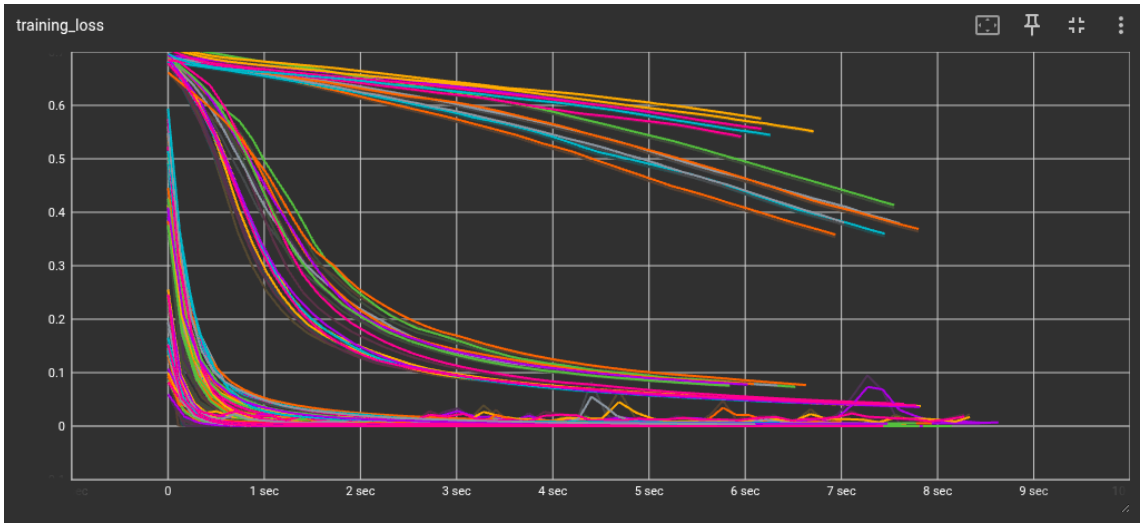


Figure 6.2: Training loss curve for the 1HSoftPlus model

Figure 6.2 illustrates the training loss during each epoch for the 1HSoftPlus model. A

decreasing trend indicates successful optimization of the model's parameters to minimize error on the training dataset.

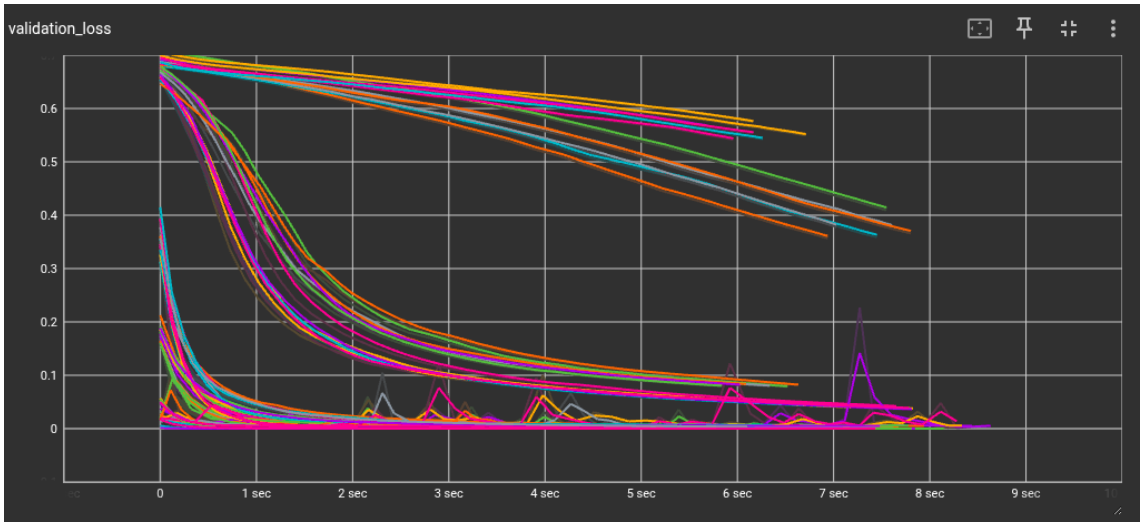


Figure 6.3: Validation loss curve for the 1HSoftPlus model

Figure 6.3 shows the validation loss throughout training. This helps evaluate how well the model generalizes to unseen data. Divergence from the training loss may indicate overfitting.

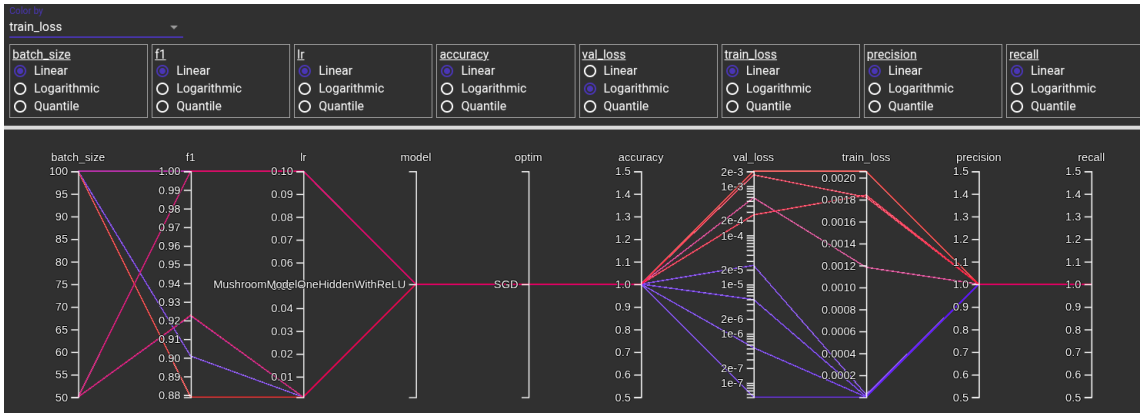


Figure 6.4: Hyperparameter influence on training loss

Figure 6.4 visualizes the impact of different hyperparameter settings on the training loss. This is useful for understanding which combinations (e.g., learning rate, batch size) lead to more efficient training and better mode